

Customer Sentiment & Intent Classifier: Feedforward Neural Networks vs. No-Code Heuristics

Stephen Rainiel Adame
ITC C508-4011
Jose Rizal University
Mandaluyong City, Philippines
stephenrainiel.adame@my.jru.edu

Abstract— This technical report evaluates the mechanics of supervised neural text classification by comparing a Python-based Keras Feedforward Neural Network to a no-code, rule-based baseline. Utilizing a customer sentiment and intent classification experiment, the study addresses the engineering limitations of hardcoded conditionals and automated heuristics. Across 21 iterative Keras experiments, early trials on a limited dataset exhibited severe instability, resulting in dramatic overfitting and underfitting. However, by executing a stepwise dataset expansion (scaling up to 345 rows) alongside learning rate optimization, the Keras model achieved an 84.62% validation accuracy, successfully reaching an Ideal Fit in EXP-18. The findings demonstrate that while granular hyperparameter tuning is critical, architectural capacity must be strictly balanced with representative dataset volume to prevent spurious correlations and memorization. Furthermore, this report details the precise location of feature extraction, forward propagation, and weight updates within a strict, non-pipelined neural architecture.

I. INTRODUCTION & PT-P1 BASELINE PERFORMANCE

Enterprise customer feedback channels are characterized by high variability, informal grammar, and subtle semantic modifiers like sarcasm. Traditional triage systems relying on static keywords and explicit IF/ELSE conditionals suffer from severe scalability and software engineering challenges. Hardcoded rules exhibit contextual fragility (e.g., keyword collisions on sarcastic phrases like "Oh great, another crash") and lack probability scoring for downstream manual triage.

In the PT-P1 baseline experiment, a no-code Intent Classifier was implemented to categorize text into Positive_Praise, Negative_Complaint, and Urgent_Report. The no-code model demonstrated that neural networks are highly sensitive to dataset sparsity. Early trials with only 3 examples per class resulted in significant fluctuations, misclassifications, and unstable confidence scoring. Stabilizing the inferences required diversifying the data with 10 to 15 representative examples. This established a critical baseline: deep learning accuracy heavily depends on the quality and quantity of the training data.

II. EXPERIMENTAL METHODOLOGY & HYPERPARAMETER CONFIGURATIONS

For the PT-P2 Keras implementation, a Sequential Feedforward Neural Network was constructed. To maintain explicit architectural control, pipeline wrapping functions were completely avoided. All data tokenization and vectorization transformations were executed as distinct, independent steps before feeding the numerical arrays into the model.

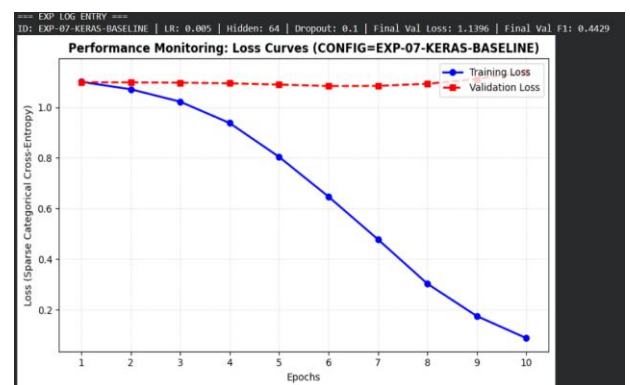
The architecture utilized an Embedding layer, a GlobalAveragePooling1D layer, a Dense hidden layer (ReLU activation), a Dropout layer for regularization, and a Dense output layer (Softmax activation).

Run ID	Input Features	Hidden Layers	Neurons per Layer	Output Neurons	Activation Function	Learning Rate	Optimizer	Loss Function	Epochs	Batch Size
EXP-01-KERAS-#1	135	1	64	3	ReLU	0.005	Adam	Cross Entropy	15	32
EXP-02-KERAS-#2	135	1	64	3	ReLU	0.003	Adam	Cross Entropy	25	32
EXP-03-KERAS-#3	135	1	64	3	ReLU	0.003	SGD	Cross Entropy	25	32
EXP-04-KERAS-#4	135	1	64	3	ReLU	0.005	SGD	Cross Entropy	25	32
EXP-05-KERAS-#5	135	1	128	3	ReLU	0.005	SGD	Cross Entropy	25	32
EXP-06-KERAS-#6	135	1	64	3	ReLU	0.005	Adam	Cross Entropy	22	8
EXP-07-KERAS-#7	135	1	64	3	ReLU	0.005	Adam	Cross Entropy	10	8
EXP-08-KERAS-#8	585	1	64	3	ReLU	0.005	Adam	Cross Entropy	10	8
EXP-09-KERAS-#9	585	1	64	3	ReLU	0.005	Adam	Cross Entropy	10	8
EXP-10-KERAS-#10	585	1	64	3	ReLU	0.002	Adam	Cross Entropy	10	8
EXP-11-KERAS-#11	585	1	64	3	ReLU	0.002	Adam	Cross Entropy	25	32
EXP-12-KERAS-#12	585	1	64	3	ReLU	0.002	SGD	Cross Entropy	25	32
EXP-13-KERAS-#13	585	1	64	3	ReLU	0.01	SGD	Cross Entropy	25	32
EXP-14-KERAS-#14	585	1	128	3	ReLU	0.001	Adam	Cross Entropy	60	32
EXP-15-KERAS-#15	1035	1	128	3	ReLU	0.001	Adam	Cross Entropy	12	32
EXP-16-KERAS-#16	1035	1	128	3	ReLU	0.001	Adam	Cross Entropy	12	24
EXP-17-KERAS-#17	1035	1	64	3	ReLU	0.002	Adam	Cross Entropy	12	32
EXP-18-KERAS-#18	1035	1	64	3	ReLU	0.002	Adam	Cross Entropy	8	32
EXP-19-KERAS-#19	1035	1	64	3	ReLU	0.002	Adam	Cross Entropy	8	31
EXP-20-KERAS-#20	1035	1	64	3	ReLU	0.002	Adam	Cross Entropy	9	31
EXP-21-KERAS-#21	1035	1	64	3	ReLU	0.002	Adam	Cross Entropy	8	30

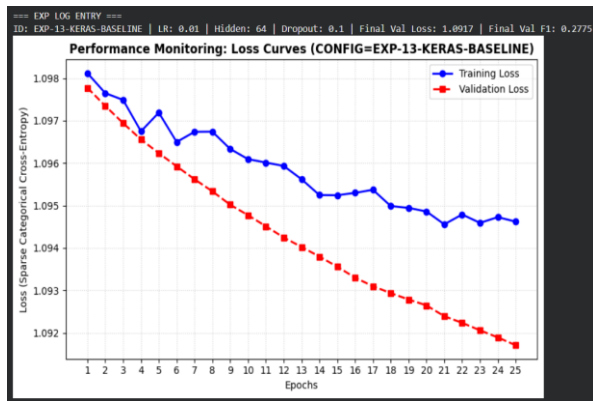
III. PERFORMANCE MONITORING & LOSS CURVE ANALYSIS

The progression across the 21 runs reveals three distinct phases of model adaptation:

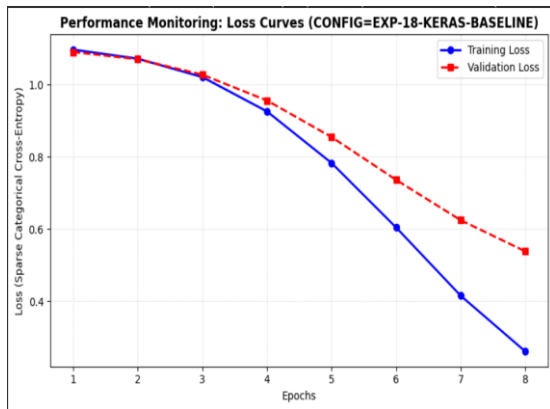
- Phase 1: Starvation & Memorization (EXP-01 to EXP-07).** On the baseline dataset, the model violently oscillated between extremes. Runs 01, 02, and 06 saw the training accuracy rapidly hit 100%, indicating the network perfectly memorized the small batch of training samples. Concurrently, validation accuracy stalled below 43%, confirming severe overfitting. Attempts to mitigate this by expanding the hidden layer to 128 neurons (EXP-05) only confused the network, collapsing it into underfitting.



- Phase 2: Initial Augmentation (EXP-08 to EXP-14).** Expanding the dataset to 195 rows yielded immediate improvements. The validation accuracy climbed consistently into the 70–75% range (EXP-08, EXP-09). However, a persistent generalization gap remained, leaving the model technically overfitted despite the improved validation scores.



3. **Phase 3: Stabilization & Ideal Fit (EXP-14 to EXP-21).** A second augmentation brought the dataset to 345 rows. When paired with a meticulously tuned learning rate of 0.002 and 64 hidden neurons, the network reached equilibrium. In **EXP-18**, training accuracy held at 99.59% while validation accuracy peaked at 84.62%. The resulting convergence bridged the generalization gap, satisfying the conditions for an Ideal Fit.



IV. DISCUSSION & DEBRIEF ANSWERS

1. **Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)** - Relying on the automated heuristics of a no-code tool (as seen in PT-P1) abstracts away the mathematical mechanics of the neural network. Feature extraction and optimization happen automatically via a single button click. In contrast, directly manipulating Keras requires an explicit understanding of the bias-variance tradeoff. The primary trade-off is control versus complexity: no-code platforms offer rapid deployment but lack the granularity to fix specific convergence failures. Code-based optimization allows precise architectural sizing and learning rate adjustments but demands rigorous manual evaluation to prevent gradient vanishing or explosion.
2. **Diagnosing Model Fit via Loss Curves & Regularization (Pillar 1 & Pillar 3)** - Overfitting in the early phases (EXP-01, 02) was diagnosed by observing a perfect training accuracy paired with a stagnant, exceptionally low validation accuracy. The network memorized the noise of the training data. To resolve this, structural capacities were adjusted (e.g., experimenting with 64 vs. 128 hidden_dim neurons)

and learning rates were tweaked. However, because the initial dataset was fundamentally too small, tuning hyperparameters alone merely caused the network to bounce between overfitting and severe underfitting (EXP-03 to EXP-05). Architectural tuning cannot replace missing data.

3. **Data Volume & Diversity vs. Architectural Capacity** - As established in the PT-P1 baseline, training a neural network on a handful of samples causes it to overfit vocabulary tokens and generate spurious correlations. Neural networks act as statistical function approximators. When the Python dataset was organically expanded in steps (first to 195 rows, then to 345 rows), the model's validation accuracy climbed from 28% to an Ideal Fit of 84.62% (EXP-18). Architectural capacity must be fed enough representative data so that the network learns contextual semantics rather than memorizing a few specific phrases
4. **Connecting Code Implementation to Slide Architecture (Pipeline Mapping)**

Within the strict Python implementation—expressly avoiding black-box pipeline wrapping functions—the architecture maps exactly to the theoretical computational graph:

- **Feature Extraction:** Raw text cannot be processed by dense layers. The data undergoes tokenization, normalization, and vectorization. In the Keras script, this is executed by the Embedding layer (which projects discrete tokens into dense vector space) and the GlobalAveragePooling1D layer.
- **Forward Pass / Loss Calculation:** This occurs implicitly inside the model.fit() loop. The numerical inputs propagate through the Dense layer to output a Softmax probability. The loss calculation is explicitly defined during compilation as loss="sparse_categorical_crossentropy".
- **Backward Pass / Weight Updates:** The backpropagation engine is controlled by the configured optimizer (e.g., opt = Adam(learning_rate=0.002) in EXP-18). After calculating the loss for a specific batch_size, the optimizer automatically computes gradients and adjusts the mathematical weights in the Dense layers to minimize error for the next epoch.

V. CONCLUSION

Supervised text classification models are structurally brittle without sufficient representative data. While hardcoded IF/ELSE rules fail to parse semantic nuances like sarcasm, neural networks remain heavily sensitive to data sparsity. As demonstrated across the PT-P1 baseline and the 21 iterations of PT-P2, deep learning optimization requires a holistic approach. Engineering teams must build diverse datasets, test against edge cases, and explicitly tune architectural capacity to achieve robust, scalable generalizations in production environments.